Chapter 4. Dependency Management

Python programmers benefit from a rich ecosystem of third-party libraries and tools. Standing on the shoulders of giants comes at a price: the packages you depend on for your projects generally depend on a number of packages themselves. All of these are moving targets—as long as any project is alive, its maintainers will publish a stream of releases to fix bugs, add features, and adapt to the evolving ecosystem.

Managing dependencies is a major challenge when you maintain software over time. You need to keep your project up-to-date, if only to close security vulnerabilities in a timely fashion. Often this requires updating your dependencies to the latest version—few open source projects have the resources to distribute security updates for older releases. You'll be updating dependencies all the time! Making the process as frictionless, automated, and reliable as possible comes with a huge payoff.

Dependencies of a Python project are the third-party packages that must be installed in its environment.[1] Most commonly, you incur a dependency on a package because it distributes a module you import. We also say that the project *requires* a package.

Many projects also use third-party tools for developer tasks—like running the test suite or building documentation. These packages are known as *development dependencies*: end users don't need them to run your code. A related case is the build dependencies from Chapter 3, which let you create packages for your project.

Dependencies are like relatives. If you depend on a package, its dependencies are your dependencies, too—no matter how much you like them.

These packages are known as *indirect dependencies*; you can think of them as a tree with your project at its root.

This chapter explains how to manage dependencies effectively. In the next section, you'll learn how to specify dependencies in *pyproject.toml* as part of the project metadata. Afterward, I'll talk about development dependencies and requirements files. Finally, I'll explain how you can *lock* dependencies to precise versions for reliable deployments and repeatable checks.

Adding Dependencies to the Example Application

As a working example, let's enhance `random-wikipedia-article` from Example 3-1 with the HTTPX library, a fully featured HTTP client that supports both synchronous and asynchronous requests as well as the newer (and far more efficient) protocol version HTTP/2. You'll also improve the output of the program using Rich, a library for rich text and beautiful formatting in the terminal.

Consuming an API with HTTPX

Wikipedia asks developers to set a `User-Agent` header with contact details. That's not so they can send out postcards to congratulate folks on their proficient use of the Wikipedia API. It gives them a way to reach out if a client inadvertently hammers their servers.

Example 4-1 shows how you can use `httpx` to send a request to the Wikipedia API with the header. You could also use the standard library to send a `User-Agent` header with your requests. But `httpx` offers a more intuitive, explicit, and flexible interface, even when you're not using any of its advanced features.

Example 4-1. Using `httpx` to consume the Wikipedia API

```
import textwrap
import httpx
```

```
API_URL = "https://en.wikipedia.org/api/rest_v1/page/random/summary"
USER_AGENT = "random-wikipedia-article/0.1 (Contact: you@example.com)"

def main():
    headers = {"User-Agent": USER_AGENT}

    with httpx.Client(headers=headers) as client: ❶
        response = client.get(API_URL, follow_redirects=True) ❷
        response.raise_for_status() ❸
        data = response.json() ❹

    print(data["title"], end="\n\n")
    print(textwrap.fill(data["extract"]))
```

❶ When creating a client instance, you can specify headers that it should send with every request—like the `User-Agent` header. Using the client as a context manager ensures that the network connection is closed at the end of the `with` block.

❷ This line performs two HTTP `GET` requests to the API. The first one goes to the *random* endpoint, which responds with a redirect to the actual article. The second one follows the redirect.

❸ The `raise_for_status` method raises an exception if the server response indicates an error via its status code.

❹ The `json` method abstracts the details of parsing the response body as JSON.

Console Output with Rich

While you're at it, let's improve the look and feel of the program. Example 4-2 uses Rich, a library for console output, to display the article title in bold. That hardly scrapes the surface of Rich's formatting options. Modern terminals are surprisingly capable, and Rich lets you leverage their potential with ease. Take a look at its official documentation for details.

Example 4-2. Using Rich to enhance console output

```
import httpx
from rich.console import Console

def main():
    ...
    console = Console(width=72, highlight=False) ❶
    console.print(data["title"], style="bold", end="\n\n") ❷
    console.print(data["extract"])
```

❶ Console objects provide a featureful `print` method for console output. Setting the console width to 72 characters replaces our earlier call to `textwrap.fill`. You'll also want to disable automatic syntax highlighting, since you're formatting prose rather than data or code.

❷ The `style` keyword allows you to set the title apart using a bold font.

Specifying Dependencies for a Project

If you haven't done so yet, create and activate a virtual environment for the project, and perform an editable install from the current directory:

```
$ uv venv
$ uv pip install --editable .
```

You may be tempted to install `httpx` and `rich` manually into the environment. Instead, add them to the project dependencies in *pyproject.toml*. This ensures that whenever you install your project, the two packages are installed along with it:

```
[project]
name = "random-wikipedia-article"
version = "0.1"
```

```
dependencies = ["httpx", "rich"]
...
```

If you reinstall the project, you'll see that uv installs its dependencies as well:

```
$ uv pip install --editable .
```

Each entry in the `dependencies` field is a *dependency specification*. Besides the package name, it lets you supply additional information: version specifiers, extras, and environment markers. The following sections explain what these are.

Version Specifiers

Version specifiers define the range of acceptable versions for a package. When you add a new dependency, it's a good idea to include its current version as a lower bound—unless your project needs to be compatible with older releases. Update the lower bound whenever you start relying on newer features of the package:

```
[project]
dependencies = ["httpx>=0.27.0", "rich>=13.7.1"]
```

Why declare lower bounds on your dependencies? Installers choose the latest version for a dependency by default. There are three reasons why you should care. First, libraries are typically installed alongside other packages, which may have additional version constraints. Second, even applications aren't always installed in isolation—for example, Linux distros may package your application for the system-wide environment. Third, lower bounds help you detect version conflicts in your own dependency tree—such as when you require a recent release of a package, but another dependency only works with its older releases.

Avoid speculative upper version bounds—you shouldn't guard against newer releases unless you know they're incompatible with your project. See "Upper Version Bounds in Python" about issues with version capping.

Lock files are a much better solution to dependency-induced breakage than upper bounds—they request "known good" versions of your dependencies when deploying a service or running automated checks (see "Locking Dependencies").

If a botched release breaks your project, publish a bugfix release to exclude that specific broken version:

```
[project]
dependencies = ["awesome>=1.2,!=1.3.1"]
```

Use an upper bound as a last resort if a dependency breaks compatibility permanently. Lift the version cap once you're able to adapt your code:

```
[project]
dependencies = ["awesome>=1.2,<2"]
```

WARNING

Excluding versions after the fact has a pitfall that you need to be aware of. Dependency resolvers can decide to downgrade your project to a version without the exclusion and upgrade the dependency anyway. Lock files can help with this.

UPPER VERSION BOUNDS IN PYTHON

Some people routinely include upper bounds in version constraints. Packages following the Semantic Versioning scheme use major versions to signal a *breaking change*: an incompatible change to their public API.

As engineers, we err on the side of safety to build robust products. At first glance, guarding against major releases seems like something any responsible person would do. Even if most of them don't break your project, isn't it better to opt in after you have a chance to test the release?

Unfortunately, upper version bounds quickly lead to unsolvable dependency conflicts.[2] Python environments (unlike Node.js environments, in particular) can contain only a single version of each package. Libraries that put upper bounds on their dependencies prevent downstream projects from receiving security and bug fixes. Before adding an upper version bound, carefully consider the costs and benefits.

What exactly constitutes a breaking change is less defined than it may seem. For example, should a project increment its major version every time it drops support for an old Python version?

Even in clear cases, a breaking change will break your project only if it affects the part of the public API that your project uses. By contrast, many changes that will break your project aren't marked by a version number: they're just bugs. In the end, you'll still rely on automated tests to discover "bad" versions and deal with them after the fact.

Version specifiers support several operators, as shown in Table 4-1. In short, use the equality and comparison operators you know from Python: `==`, `!=`, `<=`, `>=`, `<`, and `>`.

Table 4-1. Version specifiers

Operator	Name	Description
`==`	Version matching	Versions must compare equal after normalization. Trailing zeros are stripped off.
`!=`	Version exclusion	The inverse of the `==` operator
`<=`, `>=`	Inclusive ordered comparison	Performs lexicographical comparison. Prereleases precede final releases.
`<`, `>`	Exclusive ordered comparison	Similar to inclusive, but the versions must not compare equal
`~=`	Compatible release	Equivalent to `>=x.y,==x.*` to the specified precision
`===`	Arbitrary equality	Simple string comparison for nonstandard versions

Three operators merit additional discussion:

- The `==` operator supports wildcards (`*`), albeit only at the end of the version string. In other words, you can require the version to match a particular prefix, such as `1.2.*`.
- The `===` operator lets you perform a simple character-by-character comparison. It's best used as a last resort for nonstandard versions.
- The `~=` operator for compatible releases specifies that the version should be greater than or equal to the given value, while still starting with the same prefix. For example, `~=1.2.3` is equivalent to `>=1.2.3,==1.2.*`, and `~=1.2` is equivalent to `>=1.2,==1.*`.

You don't need to guard against prereleases—version specifiers exclude them by default. Prereleases are valid candidates in three situations only: when they're already installed, when no other releases satisfy the dependency specification, and when you request them explicitly, using a clause like `>=1.0.0rc1`.

Extras

Suppose you want to use the newer HTTP/2 protocol with `httpx`. This requires only a small change to the code that creates the HTTP client:

```
def main():
    headers = {"User-Agent": USER_AGENT}
    with httpx.Client(headers=headers, http2=True) as client:
        ...
```

Under the hood, `httpx` delegates the gory details of speaking HTTP/2 to another package, `h2`. That dependency is not pulled in by default, however. This way, users who don't need the newer protocol get away with a smaller dependency tree. You do need it here, so activate the optional feature using the syntax `httpx[http2]`:

```
[project]
dependencies = ["httpx[http2]>=0.27.0", "rich>=13.7.1"]
```

Optional features that require additional dependencies are known as *extras*, and you can have more than one. For example, you could specify `httpx[http2,brotli]` to allow decoding responses with *Brotli compression*, which is a compression algorithm developed at Google that's common in web servers and content delivery networks.

Optional dependencies

Let's look at this situation from the point of view of `httpx`. The `h2` and `brotli` dependencies are optional, so `httpx` declares them under `optional-dependencies` instead of `dependencies` (Example 4-3).

Example 4-3. Optional dependencies of `httpx` (simplified)

```
[project]
name = "httpx"

[project.optional-dependencies]
```

```
http2 = ["h2>=3,<5"]
brotli = ["brotli"]
```

The `optional-dependencies` field is a TOML table. It can hold multiple lists of dependencies, one for each extra provided by the package. Each entry is a dependency specification and uses the same rules as the `dependencies` field.

If you add an optional dependency to your project, how do you use it in your code? Don't check if your package was installed with the extra—just import the optional package. You can catch the `ImportError` exception if the user didn't request the extra:

```
try:
    import h2
except ImportError:
    h2 = None

# Check h2 before use.
if h2 is not None:
    ...
```

This is a common pattern in Python—so common it comes with a name and an acronym: "Easier to Ask Forgiveness than Permission" (EAFP). Its less Pythonic counterpart is dubbed "Look Before You Leap" (LBYL).

Environment Markers

The third piece of metadata you can specify for a dependency is environment markers. Before I explain what these markers are, let me show you an example of where they come in handy.

If you looked at the `User-Agent` header in Example 4-1 and thought, "I shouldn't have to repeat the version number in the code," you're absolutely right. As you saw in "Single-Sourcing the Project Version", you can read the version of your package from its metadata in the environment.

Example 4-4 shows how you can use the function `importlib.metadata.metadata` to construct the `User-Agent` header from the core metadata fields `Name`, `Version`, and `Author-email`. These fields correspond to `name`, `version`, and `authors` in the project metadata.[3]

Example 4-4. Using `importlib.metadata` to build a `User-Agent` header

```
from importlib.metadata import metadata

USER_AGENT = "{Name}/{Version} (Contact: {Author-email})"

def build_user_agent():
    fields = metadata("random-wikipedia-article") ❶
    return USER_AGENT.format_map(fields) ❷

def main():
    headers = {"User-Agent": build_user_agent()}
    ...
```

❶ The `metadata` function retrieves the core metadata fields for the package.

❷ The `str.format_map` function looks up each placeholder in the mapping.

The `importlib.metadata` library was introduced in Python 3.8. While it's now available in all supported versions, that wasn't always so. Were you out of luck if you had to support an older Python version?

Not quite. Fortunately, many additions to the standard library come with *backports*—third-party packages that provide the functionality for older interpreters. For `importlib.metadata`, you can fall back to the `importlib-metadata` backport from PyPI. The backport remains useful because the library changed several times after its introduction.

You need backports only in environments that use specific Python versions. An environment marker lets you express this as a conditional

dependency:

```
importlib-metadata; python_version < '3.8'
```

Installers will install the package only on interpreters older than Python 3.8.

More generally, an *environment marker* expresses a condition that an environment must satisfy for the dependency to apply. Installers evaluate the condition on the interpreter of the target environment.

Environment markers let you request dependencies for specific operating systems, processor architectures, Python implementations, or Python versions. Table 4-2 lists all the environment markers at your disposal, as specified in PEP 508.[4]

Table 4-2. Environment markers[a]

Environment marker	Standard library	Description	Examples
`os_name`	`os.name()`	The operating system family	`posix`, `nt`
`sys_platform`	`sys.platform()`	The platform identifier	`linux`, `darwin`, `win32`
`platform_system`	`platform.system()`	The system name	`Linux`, `Darwin`, `Windows`
`platform_release`	`platform.release()`	The operating system release	`23.2.0`
`platform_version`	`platform.version()`	The system release	`Darwin Kernel Version 23.2.0: ...`
`platform_machine`	`platform.machine()`	The processor architecture	`x86_64`, `arm64`
`python_version`	`platform.python_version_tuple()`	The Python feature version in the format `x.y`	`3.12`
`python_full_version`	`platform.python_version()`	The full Python version	`3.12.0`, `3.13.0a4`
`platform_python_implementation`	`platform.python_implementation()`	The Python implementation	`CPython`, `PyPy`
`implementation_name`	`sys.implementation.name`	The Python implementation	`cpython`, `pypy`
`implementation_version`	`sys.implementation.version`	The Python implementation version	`3.12.0`, `3.13.0a4`

[a] The `python_version` and `implementation_version` markers apply transformations. See PEP 508 for details.

Going back to Example 4-4, here are the `requires-python` and `dependencies` fields to make the package compatible with Python 3.7:

```
[project]
requires-python = ">=3.7"
dependencies = [
    "httpx[http2]>=0.24.1",
    "rich>=13.7.1",
    "importlib-metadata>=6.7.0; python_version < '3.8'",
]
```

The import name for the backport is `importlib_metadata`, while the standard library module is named `importlib.metadata`. You can import the appropriate module in your code by checking the Python version in `sys.version_info`:

```
if sys.version_info >= (3, 8):
    from importlib.metadata import metadata
else:
    from importlib_metadata import metadata
```

Did I just hear somebody shout "EAFP"? If your imports depend on the Python version, it's better to avoid the technique from "Optional dependencies" and "look before you leap." An explicit version check communicates your intent to static analyzers, such as the mypy type checker (see Chapter 10). EAFP may result in errors from these tools because they can't detect when each module is available.

Markers support the same equality and comparison operators as version specifiers (Table 4-1). Additionally, you can use `in` and `not in` to match a substring against the marker. For example, the expression `'arm' in platform_version` checks if `platform.version()` contains the string `'arm'`.

You can also combine multiple markers using the Boolean operators `and` and `or`. Here's a rather contrived example combining all these features:

```
[project]
dependencies = ["""                                                    \
  awesome-package; python_full_version <= '3.8.1'                      \
    and (implementation_name == 'cpython' or implementation_name == 'pypy') \
    and sys_platform == 'darwin'                                       \
    and 'arm' in platform_version                                      \
"""]
```

The example also relies on TOML's support for multiline strings, which uses triple quotes just like Python. Dependency specifications cannot span multiple lines, so you have to escape the newlines with a backslash.

Development Dependencies

Development dependencies are third-party packages that you require during development. As a developer, you might use the pytest testing framework to run the test suite for your project, the Sphinx documentation system to build its docs, or a number of other tools to help with project maintenance. Your users, on the other hand, don't need to install any of these packages to run your code.

An Example: Testing with pytest

As a concrete example, let's add a small test for the `build_user_agent` function from Example 4-4. Create a directory *tests* with two files: an empty *__init__.py* and a module *test_random_wikipedia_article.py* with the code from Example 4-5.

Example 4-5. Testing the generated `User-Agent` header

```
from random_wikipedia_article import build_user_agent

def test_build_user_agent():
    assert "random-wikipedia-article" in build_user_agent()
```

Example 4-5 uses only built-in Python features, so you could just import and run the test manually. But even for this tiny test, pytest adds three

useful features. First, it discovers modules and functions whose names start with `test`, so you can run your tests by invoking `pytest` without arguments. Second, pytest shows tests as it executes them, as well as a summary with the test results at the end. Third, pytest rewrites assertions in your tests to give you friendly, informative messages when they fail.

Let's run the test with pytest. I'm assuming you already have an active virtual environment with an editable install of your project. Enter the commands below to install and run pytest in that environment:

```
$ uv pip install pytest
$ py -m pytest
========================= test session starts ==========================
platform darwin -- Python 3.12.2, pytest-8.1.1, pluggy-1.4.0
rootdir: ...
plugins: anyio-4.3.0
collected 1 item

tests/test_random_wikipedia_article.py .                          [100%]

========================== 1 passed in 0.22s ===========================
```

For now, things look great. Tests help your project evolve without breaking things. The test for `build_user_agent` is a first step in that direction. Installing and running pytest is a small infrastructure cost compared to these long-term benefits.

Setting up a project environment becomes harder as you acquire more development dependencies—documentation generators, linters, code formatters, type checkers, or other tools. Even your test suite may require more than pytest: plugins for pytest, tools for measuring code coverage, or just packages that help you exercise your code.

You also need compatible versions of these packages—your test suite may require the latest version of pytest, while your documentation may not build on the new Sphinx release. Each of your projects may have slightly different requirements. Multiply this by the number of developers working on each project, and it becomes clear that you need a way to track your development dependencies.

As of this writing, Python doesn't have a standard way to declare the development dependencies of a project—although many Python project managers support them in their `[tool]` table and a draft PEP exists.[5] Besides project managers, people use two approaches to fill the gap: optional dependencies and requirements files.

Optional Dependencies

As you've seen in "Extras", the `optional-dependencies` table contains groups of optional dependencies named extras. It has three properties that make it suitable for tracking development dependencies. First, the packages aren't installed by default, so end users don't pollute their Python environment with them. Second, it lets you group the packages under meaningful names like `tests` or `docs`. And third, the field comes with the full expressivity of dependency specifications, including version constraints and environment markers.

On the other hand, there's an impedance mismatch between development dependencies and optional dependencies. Optional dependencies are exposed to users through the package metadata—they let users opt into features that require additional packages. By contrast, users aren't meant to install development dependencies—these packages aren't required for any user-facing features.

Furthermore, you can't install extras without the project itself. By contrast, not all developer tools need your project installed. For example, linters analyze your source code for bugs and potential improvements. You can run them on a project without installing it into the environment. Besides wasting time and space, "fat" environments constrain dependency resolution unnecessarily. For example, many Python projects could no longer upgrade important dependencies when the Flake8 linter put a version cap on `importlib-metadata`.

Keeping this in mind, extras are widely used for development dependencies and are the only method covered by a packaging standard. They're a pragmatic choice, especially if you manage linters with pre-commit (see Chapter 9). Example 4-6 shows how you'd use extras to track packages required for testing and documentation.

Example 4-6. Using extras to represent development dependencies

```
[project.optional-dependencies]
tests = ["pytest>=7.4.4", "pytest-sugar>=1.0.0"] ❶
docs = ["sphinx>=5.3.0"] ❷
```

❶ The `pytest-sugar` plugin enhances pytest's output with a progress bar and shows failures immediately.

❷ Sphinx is a documentation generator used by the official Python documentation and many open source projects.

You can now install the test dependencies using the `tests` extra:

```
$ uv pip install -e ".[tests]"
$ py -m pytest
```

You can also define a `dev` extra with all the development dependencies. This lets you set up a development environment in one go, with your project and every tool it uses:

```
$ uv pip install -e ".[dev]"
```

There's no need to repeat all the packages when you define `dev`. Instead, you can just reference the other extras, as shown in Example 4-7.

Example 4-7. Providing a `dev` extra with all development dependencies

```
[project.optional-dependencies]
tests = ["pytest>=7.4.4", "pytest-sugar>=1.0.0"]
docs = ["sphinx>=5.3.0"]
dev = ["random-wikipedia-article[tests,docs]"]
```

This style of declaring an extra is also known as a *recursive optional dependency*, since the package with the `dev` extra depends on itself (with `tests` and `docs` extras).

Requirements Files

Requirements files are plain text files with dependency specifications on each line (Example 4-8). Additionally, they can contain URLs and paths, optionally prefixed by `-e` for an editable install, as well as global options, such as `-r` to include another requirements file or `--index-url` to use a package index other than PyPI. The file format also supports Python-style comments (with a leading `#` character) and line continuations (with a trailing `\` character).

Example 4-8. A simple requirements.txt file

```
pytest>=7.4.4
pytest-sugar>=1.0.0
sphinx>=5.3.0
```

You can install the dependencies listed in a requirements file using pip or uv:

```
$ uv pip install -r requirements.txt
```

By convention, a requirements file is named *requirements.txt*. However, variations are common. You might have a *dev-requirements.txt* for development dependencies or a *requirements* directory with one file per dependency group (Example 4-9).

Example 4-9. Using requirements files to specify development dependencies

```
# requirements/tests.txt
-e . ❶
pytest>=7.4.4
pytest-sugar>=1.0.0

# requirements/docs.txt
sphinx>=5.3.0 ❷

# requirements/dev.txt
```

```
-r tests.txt ❸
-r docs.txt
```

❶ The *tests.txt* file requires an editable install of the project because the test suite needs to import the application modules.

❷ The *docs.txt* file doesn't require the project. (That's assuming you build the documentation from static files only. If you use the `autodoc` Sphinx extension to generate API documentation from docstrings in your code, you'll also need the project here.)

❸ The *dev.txt* file includes the other requirements files.

NOTE

If you include other requirements files using `-r`, their paths are evaluated relative to the including file. By contrast, paths to dependencies are evaluated relative to your current directory, which is typically the project directory.

Create and activate a virtual environment, then run the following commands to install the development dependencies and run the test suite:

```
$ uv pip install -r requirements/dev.txt
$ py -m pytest
```

Requirements files aren't part of the project metadata. You share them with other developers using the version control system, but they're invisible to your users. For development dependencies, this is exactly what you want. What's more, requirements files don't implicitly include your project in the dependencies. That shaves time from all tasks that don't need the project installed.

Requirements files also have downsides. They aren't a packaging standard and are unlikely to become one—each line of a requirements file is essentially an argument to `pip install`. "Whatever pip does" may remain the unwritten law for many edge cases in Python packaging, but community standards replace it more and more. Another downside is the

clutter these files cause in your project when compared to a table in *pyproject.toml*.

As mentioned above, Python project managers let you declare development dependencies in *pyproject.toml*, outside of the project metadata—Rye, Hatch, PDM, and Poetry all offer this feature. See Chapter 5 for a description of Poetry's dependency groups.

Locking Dependencies

You've installed your dependencies in a local environment or in continuous integration (CI), and you've run your test suite and any other checks you have in place. Everything looks good, and you're ready to deploy your code. But how do you install the same packages in production that you used when you ran your checks?

Using different packages in development and production has consequences. Production may end up with a package that's incompatible with your code, has a bug or security vulnerability, or—in the worst case—has been hijacked by an attacker. If your service gets a lot of exposure, this scenario is worrying—and it can involve any package in your dependency tree, not just those that you import directly.

WARNING

Supply chain attacks infiltrate a system by targeting its third-party dependencies. For example, in 2022, a threat actor dubbed "JuiceLedger" uploaded malicious packages to legitimate PyPI projects after compromising them with a phishing campaign.[6]

There are many reasons why environments end up with different packages given the same dependency specifications. Most of them fall into two categories: upstream changes and environment mismatch. First, you can get different packages if the set of available packages changes upstream:

- A new release comes in before you deploy.

- A new artifact is uploaded for an existing release. For example, maintainers sometimes upload additional wheels when a new Python release comes out.
- A maintainer deletes or yanks a release or artifact. *Yanking* is a soft delete that hides the file from dependency resolution unless you request it specifically.

Second, you can get different packages if your development environment doesn't match the production environment:

- Environment markers evaluate differently on the target interpreter (see "Environment Markers"). For example, the production environment might use an old Python version that requires a backport like `importlib-metadata`.
- Wheel compatibility tags can cause the installer to select a different wheel for the same package (see "Wheel Compatibility Tags"). For example, this can happen if you develop on a Mac with Apple silicon while production uses Linux on an x86-64 architecture.
- If the release doesn't include a wheel for the target environment, the installer builds it from the sdist on the fly. Wheels for extension modules often lag behind when a new Python version sees the light.
- If the environments don't use the same installer (or different versions of the same installer), each installer may resolve the dependencies differently. For example, uv uses the PubGrub algorithm for dependency resolution,[7] while pip uses a backtracking resolver for Python packages, `resolvelib`.
- Tooling configuration or state can also cause different results—for example, you might install from a different package index or from a local cache.

You need a way to define the exact set of packages required by your application, and you want its environment to be an exact image of this package inventory. This process is known as *locking*, or *pinning*, the project dependencies, which are listed in a *lock file*.

So far, I've talked about locking dependencies for reliable and reproducible deployments. Locking is also beneficial during development, for both applications and libraries. By sharing a lock file with your team and

with contributors, you put everybody on the same page: every developer uses the same dependencies when running the test suite, building the documentation, or performing other tasks. Using the lock file for mandatory checks avoids surprises where checks fail in CI after passing locally. To reap these benefits, lock files must include development dependencies, too.

As of this writing, Python lacks a packaging standard for lock files—although the topic is under active consideration.[8] Meanwhile, many Python project managers, such as Poetry, PDM, and pipenv, have implemented their own lock file formats; others, like Rye, use requirements files for locking dependencies.

In this section, I'll introduce two methods for locking dependencies using requirements files: *freezing* and *compiling requirements*. In Chapter 5, I'll describe Poetry's lock files.

"LOCKING" DEPENDENCIES IN THE PROJECT METADATA

If you want to lock the dependencies for an application, why not narrow the version constraints in *pyproject.toml*? For example, couldn't you lock the dependencies on `httpx` and `rich` as shown below?

```
[project]
dependencies = ["httpx[http2]==0.27.0", "rich==13.7.1"]
```

There are two main problems with this approach.

First, you've locked only direct dependencies. For example, `random-wikipedia-article` uses `h2` to communicate via HTTP/2, but that package is missing from the dependency specifications.

Second, and more importantly, you've lost valuable information: the compatible version ranges for your top-level dependencies. Version constraints determine the search space for a dependency resolver. The resolver can no longer help you upgrade packages or resolve dependencies for a new environment—like when you bump the Python version in production.

You need a way to record dependencies outside of the `dependencies` table.

Freezing Requirements with pip and uv

Requirements files are a popular format for locking dependencies. They let you keep the dependency information separate from the project metadata. Pip and uv can generate these files from an existing environment:

```
$ uv pip install .
$ uv pip freeze
anyio==4.3.0
certifi==2024.2.2
h11==0.14.0
h2==4.1.0
hpack==4.0.0
```

```
httpcore==1.0.4
httpx==0.27.0
hyperframe==6.0.1
idna==3.6
markdown-it-py==3.0.0
mdurl==0.1.2
pygments==2.17.2
random-wikipedia-article @ file:///Users/user/random-wikipedia-article
rich==13.7.1
sniffio==1.3.1
```

Taking an inventory of the packages installed in an environment is known as *freezing*. Store the list in *requirements.txt* and commit the file to source control—with one change: replace the file URL with a dot for the current directory. This lets you use the requirements file anywhere, as long as you're inside the project directory.

When deploying your project to production, you can install the project and its dependencies like this:

```
$ uv pip install -r requirements.txt
```

Assuming your development environment uses a recent interpreter, the requirements file won't list `importlib-metadata`—that library is only required before Python 3.8. If your production environment runs an ancient Python version, your deployment will break. There's an important lesson here: lock your dependencies in an environment that matches the production environment.

TIP

Lock your dependencies on the same Python version, Python implementation, operating system, and processor architecture as those used in production. If you deploy to multiple environments, generate a requirements file for each one.

Freezing requirements comes with a few limitations. First, you need to install your dependencies every time you refresh the requirements file. Second, it's easy to pollute the requirements file inadvertently if you tem-

porarily install a package and forget to create the environment from scratch afterward.[9] Third, freezing doesn't allow you to record package hashes—it merely takes an inventory of an environment, and environments don't record hashes for the packages you install into them. (I'll cover package hashes in the next section.)

Compiling Requirements with pip-tools and uv

The pip-tools project lets you lock dependencies without these limitations. You can compile requirements directly from *pyproject.toml*, without installing the packages. Under the hood, pip-tools leverages pip and its dependency resolver.

Pip-tools comes with two commands: `pip-compile`, to create a requirements file from dependency specifications, and `pip-sync`, to apply the requirements file to an existing environment. The uv tool provides drop-in replacements for both commands: `uv pip compile` and `uv pip sync`.

Run `pip-compile` in an environment that matches the target environment for your project. If you use pipx, specify the target Python version:

```
$ pipx run --python=3.12 --spec=pip-tools pip-compile
```

By default, `pip-compile` reads from *pyproject.toml* and writes to *requirements.txt*. You can use the `--output-file` option to specify a different destination. The tool also prints the requirements to standard error, unless you specify `--quiet` to switch off terminal output.

Uv requires you to be explicit about the input and output files:

```
$ uv pip compile --python-version=3.12 pyproject.toml -o requirements.txt
```

Pip-tools and uv annotate the file to indicate the dependent package for each dependency, as well as the command used to generate the file. There's one more difference to the output of `pip freeze`: the compiled requirements don't include your own project. You'll have to install it separately after applying the requirements file.

Requirements files allow you to specify package hashes for each dependency. These hashes add another layer of security to your deployments: they enable you to install only vetted packaging artifacts in production. The option `--generate-hashes` includes SHA-256 hashes for each package listed in the requirements file. For example, here are hashes over the sdist and wheel files for an `httpx` release:

```
httpx==0.27.0 \
--hash=sha256:71d5465162c13681bff01ad59b2cc68dd838ea1f10e51574bac27103f00c91a5 \
--hash=sha256:a0cb88a46f32dc874e04ee956e4c2764aba2aa228f650b06788ba6bda2962ab5
```

Package hashes make installations more deterministic and reproducible. They're also an important tool in organizations that require screening every artifact that goes into production. Validating the integrity of packages prevents *on-path attacks* where a threat actor ("man in the middle") intercepts a package download to supply a compromised artifact.

Hashes also have the side effect that pip refuses to install packages without them: either all packages have hashes, or none do. As a consequence, hashes protect you from installing files that aren't listed in the requirements file.

Install the requirements file in the target environment using pip or uv, followed by the project itself. You can harden the installation using a couple of options: the option `--no-deps` ensures that you only install packages listed in the requirements file, and the option `--no-cache` prevents the installer from reusing downloaded or locally built artifacts:

```
$ uv pip install -r requirements.txt
$ uv pip install --no-deps --no-cache .
```

Update your dependencies at regular intervals. Once per week may be acceptable for a mature application running in production. Daily may be more appropriate for a project under active development—or even as soon as the releases come in. Tools like Dependabot and Renovate help with this chore: they open pull requests in your repositories with automated dependency upgrades.

If you don't upgrade dependencies regularly, you may be forced to apply a "big bang" upgrade under time pressure. A single security vulnerability can force you to port your project to major releases of multiple packages, as well as Python itself.

You can upgrade your dependencies all at once, or one dependency at a time. Use the `--upgrade` option to upgrade all dependencies to their latest version, or pass a specific package with the `--upgrade-package` option (`-P`).

For example, here's how you'd upgrade Rich to the latest version:

```
$ uv pip compile -p 3.12 pyproject.toml -o requirements.txt -P rich
```

So far, you've created the target environment from scratch. You can also use `pip-sync` to synchronize the target environment with the updated requirements file. Don't install pip-tools in the target environment for this: its dependencies may conflict with those of your project. Instead, use pipx, as you did with `pip-compile`. Point `pip-sync` to the target interpreter using its `--python-executable` option:

```
$ py -m venv .venv
$ pipx run --spec=pip-tools pip-sync --python-executable=.venv/bin/python
```

The command removes the project itself since it's not listed in the requirements file. Reinstall it after synchronizing:

```
$ .venv/bin/python -m pip install --no-deps --no-cache .
```

Uv uses the environment in *.venv* by default, so you can simplify these commands:

```
$ uv pip sync requirements.txt
$ uv pip install --no-deps --no-cache .
```

In "Development Dependencies", you saw two ways to declare development dependencies: extras and requirements files. Pip-tools and uv support both as inputs. If you track development dependencies in a `dev` extra, generate the *dev-requirements.txt* file like this:

```
$ uv pip compile --extra=dev pyproject.toml -o dev-requirements.txt
```

If you have finer-grained extras, the process is the same. You may want to store the requirements files in a *requirements* directory to avoid clutter.

If you specify your development dependencies in requirements files instead of extras, compile each of these files in turn. By convention, input requirements use the *.in* extension, while output requirements use the *.txt* extension (Example 4-10).

Example 4-10. Input requirements for development dependencies

```
# requirements/tests.in
pytest>=7.4.4
pytest-sugar>=1.0.0

# requirements/docs.in
sphinx>=5.3.0

# requirements/dev.in
-r tests.in
-r docs.in
```

Unlike Example 4-9, the input requirements don't list the project itself. If they did, the output requirements would include the path to the project—and every developer would end up with a different path. Instead, pass *pyproject.toml* together with the input requirements to lock the entire set of dependencies together:

```
$ uv pip compile requirements/tests.in pyproject.toml -o requirements/tests.txt
$ uv pip compile requirements/docs.in -o requirements/docs.txt
$ uv pip compile requirements/dev.in pyproject.toml -o requirements/dev.txt
```

Remember to install the project after you've installed the output requirements.

Why bother compiling *dev.txt* at all? Can't it just include *docs.txt* and *tests.txt*? If you install separately locked requirements on top of each other, they may well end up conflicting. Let the dependency resolver see the full picture. If you pass all the input requirements, it can give you a consistent dependency tree in return.

Table 4-3 summarizes the command-line options for `pip-compile` (and `uv pip compile`) you've seen in this chapter:

Table 4-3. Selected command-line options for `pip-compile`

Option	Description
`--generate-hashes`	Include SHA-256 hashes for every packaging artifact
`--output-file`	Specify the destination file
`--quiet`	Do not print the requirements to standard error
`--upgrade`	Upgrade all dependencies to their latest version
`--upgrade-package=<package>`	Upgrade a specific package to its latest version
`--extra=<extra>`	Include dependencies from the given extra in *pyproject.toml*

Summary

In this chapter, you've learned how to declare project dependencies using *pyproject.toml* and how to declare development dependencies using either extras or requirements files. You've also learned how to lock dependencies for reliable deployments and reproducible checks using pip-tools and uv. In the next chapter, you'll see how the project manager Poetry

helps with dependency management using dependency groups and lock files.

[1] In a wider sense, the dependencies of a project consist of all software packages that users require to run its code—including the interpreter, the standard library, third-party packages, and system libraries. Conda and distro package managers like APT, DNF, and Homebrew support this generalized notion of dependencies.

[2] Henry Schreiner, "Should You Use Upper Bound Version Constraints?", December 9, 2021.

[3] For simplicity, the code doesn't handle multiple authors—which one ends up in the header is undefined.

[4] Robert Collins, "PEP 508 – Dependency specification for Python Software Packages", November 11, 2015.

[5] Stephen Rosen, "PEP 735 – Dependency Groups in pyproject.toml", November 20, 2023.

[6] Dan Goodin, "Actors Behind PyPI Supply Chain Attack Have Been Active Since Late 2021", September 2, 2022.

[7] Natalie Weizenbaum, "PubGrub: Next-Generation Version Solving", April 2, 2018.

[8] Brett Cannon, "Lock Files, Again (But This Time w/ Sdists!)", February 22, 2024.

[9] Uninstalling the package isn't enough: the installation can have side effects on your dependency tree. For example, it may upgrade or downgrade other packages or pull in additional dependencies.